Introduction to Lists and Tuples 📋

Christopher Martin - *Bowdoin College*
c.martin@bowdoin.edu

String Slices

We have seen that strings are a little different from the other primitive types...

They store multiple characters that we can access into using positions or indices ...

C	h	r	i	s	t	o	p	h	e	r
0	1	2	3	4	5	6	7	8	9	10

```
1 name: str = "Christopher"
2
3 print(name[0], name[5:9]) # Access position 0 and access from 5 up until 9
```

```
C toph
```

String Methods

We have also seen how we can use the `my_str.method()` syntax to ask questions and execute various behaviors on specific strings in using methods !

```python
name: str = "Christopher Martin"
search: str = "r"

print(f"{name} in all uppercase is {name.upper()}! Was it originally all upper? {name.isupper()}")
print(f"{name} in all lowercase is {name.lower()}! Was it originally all upper? {name.islower()}")
print(f"There are {name.count(search)} {search}'s in {name}")
if search in name:
  print(f"The first {search} is at position {name.find(search)}")

number_or_text: str = "54"
print(f"Is {number_or_text} all digits? {number_or_text.isdigit()}")
print(f"Is {number_or_text} all letters? {number_or_text.isalpha()}")
```

```
Christopher Martin in all uppercase is CHRISTOPHER MARTIN! Was it originally all upper? False
Christopher Martin in all lowercase is christopher martin! Was it originally all upper? False
There are 3 r's in Christopher Martin
The first r is at position 2
Is 54 all digits? True
```

Lists and Tuples

Lists and Tuples work in the same way but they allow us to group any combination of things together in one variable!

- **Lists are created by surrounding a comma separated sequence with brackets `[]`!**
- **Tuples are created by surrounding a sequence with parentheses `()`!**

```python
numbers: list[int] = [1, 3, 3, 7]
letters: tuple[str] = ("a", "c", "b", "a", "z")

print(numbers)
print(letters)
```

```
[1, 3, 3, 7]
('a', 'c', 'b', 'a', 'z')
```

Lists and Tuple Ordering

Lists and Tuples store things in an ordered fashion just like strings!
We can access individual items with slices and determine their length!

`[1,`	`3,`	`3,`	`7]`	`("a",`	`"c",`	`"b",`	`"a",`	`"z")`
0	1	2	3	0	1	2	3	4

```
1  numbers: list[int] = [1, 3, 3, 7]
2  letters: tuple[str] = ("a", "c", "b", "a", "z")
3
4  print(numbers[0], numbers[1:], ": numbers has", len(numbers), "elements!")
5  print(letters[1], letters[2:], ": letters has", len(letters), "elements!")
```

```
1 [3, 3, 7] : numbers has 4 elements!
c ('b', 'a', 'z') : letters has 5 elements!
```

List Operators

List support many of the same operators as strings!

```python
list_1: list[int] = [1, 3, 7]
list_2: list[str] = ["two", "four", "six"]

# We can use the in operator to check if something is in a list
if 2 in list_1:
  print("2 is in list_1!")
else:
  print("2 is not in list_1!")

# We can compare lists together!
if list_1 == list_2:
  print("These two lists contain the same elements in the same order!")
else:
  print("Something is different about these two lists!")

# We can concatenate lists together into a new list!
print(list_1 + list_2)
```

```
2 is not in list_1!
Something is different about these two lists!
[1, 3, 7, 'two', 'four', 'six']
```

Tuple Operators

Tuples support the same operators too!

```python
tuple_1: tuple[float] = (2.0, 4.0, 6.0)
tuple_2: tuple[str] = (7, 3, 1)

# We can use the in operator to check if something is in a tuple
if 2 in tuple_1:
  print("2 is in tuple_1")
else:
  print("2 is not in tuple_1!")

# We can compare tuple together!
if tuple_1 == tuple_2:
  print("These two tuples contain the same elements in the same order!")
else:
  print("Something is different about these two tuples!")

# We can concatenate tuple together into a new tuple!
print(tuple_1 + tuple_2)
```

```
2 is in tuple_1
Something is different about these two tuples!
(2.0, 4.0, 6.0, 7, 3, 1)
```

Operator Incompatibility

However, we can’t always mix lists and tuples together...

```python
my_list: list[float] = [2.0, 4.0]
my_tuple: tuple[float] = (2.0, 4.0)

# We can compare lists and tuples but they will never be equal!
if my_list == my_tuple:
  print("This can never happen 😭")
else:
  print("One being a list and the other being a tuple makes them 'not equal'!")

# We can't concatenate lists and tuples together!
print(my_list + my_tuple)
```

```
Traceback (most recent call last):
  File "/app/output.s", line 11, in <module>
    print(my_list + my_tuple)
          ~~~~~~~~^~~~~~~~~~
TypeError: can only concatenate list (not "tuple") to list
```

Wait... these seem really similar...

Why do we have both?

Lists vs Tuples

Lists

Lists are mutable collections

They can be modified after creation!

```python
my_list: list[int] = [1, 2, 3]

my_list[2] = 1
print(my_list)
```

```
[1, 2, 1]
```

Tuples

Tuples are immutable collections

They cannot be modified after creation!

```python
my_tuple: tuple[int] = (1, 2, 3)

my_tuple[2] = 1
print(my_tuple)
```

```
Traceback (most recent call last):
  File "/app/output.s", line 3, in <module>
    my_tuple[2] = 1
    ~~~~~~~~^^^
TypeError: 'tuple' object does not support item
assignment
```

List Methods - Mutable

Method	Description
`my_list.append(v)`	Adds the given value `v` to the end of `my_list`
`my_list.insert(i, v)`	Adds the given value `v` to `my_list` at the index `i`
`my_list.remove(v)`	Removes the first occurrence of the value `v` from `my_list`
`my_list.pop(i)`	Removes the element at the index `i` from `my_list`
`my_list.clear()`	Removes everything from `my_list`
`my_list.reverse()`	Reverses the order of the elements in `my_list`
`my_list.sort()`	Attempts to sort everything in `my_list` in ascending order

Tuple Methods and additional List methods

Tuples have some methods as well but nearly as many!

- **Lists also have these methods too!**

Method	Description
`list_or_tuple.index(v)`	Returns the index of the first occurrence of the value `v` in `list_or_tuple`
`list_or_tuple.count(v)`	Returns the number of occurrences of the value `v` in `list_or_tuple`